Negative-Space Intelligence

The Failure-Boundary Calculus: Designing Intelligent Systems from Absence Inward

---

The mathematical foundation for the Negative-Space Governance Engine: assumption semantics, the absence lattice, detector design, the decay of ignorance, the silence calculus, and failure-boundary design as an engineering discipline. What a system does not know is treated as a typed, composable, monitorable object — every negative-space claim tied to a detection predicate or a design artifact.

Dr. Christabel Odeta — SAVANT / CLAI-OS / AEGIS-GLOBAL
Version 1.0 · 2026-10

C0 INFORMATIONAL · MATHEMATICAL SPECIFICATION — ALL PROPERTIES STATED AS PROOF OBLIGATIONS UNTIL DISCHARGED

DOCUMENT CONTROL & EPISTEMIC CONTRACT

0.1 Provenance and Position in the Program

This is Paper 3 of the research-and-build program defined by the *SVT Architect Analysis*: the Negative-Space paper — the source's most radical proposal, that boundary conditions (mortality, uncertainty, infrastructure failure, poverty, silence, caregiver welfare, epistemic limits) be treated as *first-class computational primitives*, not exception handling. Paper 0 named the negative space and gave it a calculus of contraction; Paper 1 gave the lifecycle dynamics; Paper 2 gave composition. This paper gives absence itself a semantics: what it means for an assumption to be missing, how missingness is detected, how it compounds, and how a system designed from failure boundaries inward differs structurally from one designed outward from capability. It reconciles against all three predecessors under the standing rule: stricter constraint wins, adoptions recorded (§11).

0.2 Epistemic Contract

Unchanged: VERIFIED-FROM-SOURCE, FORMALIZED-HEREIN, CONJECTURE, UNKNOWN. All named properties are proof obligations. One sharpening for this paper's subject: negative-space claims are the easiest to overstate, because "we considered what could go wrong" is unfalsifiable in prose. Every negative-space claim here is therefore tied to a detection predicate or a design artifact; an absence that cannot be pointed to in a log or a registry entry does not count as represented.

0.3 Authority Boundary

C0 informational. The failure-boundary method of §8 is a design discipline, not a safety certification. No absence registry, however complete, establishes that a system is safe — it establishes that the system's *known* failure boundaries are enumerated, monitored, and priced into authority. The unknown unknowns remain, and §5.4 is about what the architecture owes them.

0.4 The Thesis in One Sentence

Most systems are designed from capability outward and fail at their edges; a negative-space system is designed from its edges inward — the boundaries are the specification, and capability is what fills the space they leave. This document specifies the edges.

TABLE OF CONTENTS

Part I — The Semantics of Absence

1 Absence as a Computational Primitive **4**

2 Assumption Semantics and the Registry **6**

3 The Absence Lattice **8**

Appendix RW — Related Work and Positioning 32

Part II — The Dynamics of Absence

4 Detection: Sensors for What Is Not There **10**

5 The Decay of Ignorance **12**

6 Composition of Absence **14**

Part III — The Human Absences

7 The Silence Calculus **16**

8 The Failure-Boundary Design Method **18**

Part IV — Evidence and Governance

9 Worked Absence Traces **20**

10 Falsifiable Hypotheses (H22–H28) and Verification **22**

11 Reconciliation with Papers 0–2 and PRIME **24**

12 Output Contract **25**

13 References **27**

Appendix A Absence Taxonomy Index **28**

Appendix B Registry Schema and Detector Catalog **30**

PART I — THE SEMANTICS OF ABSENCE

1 Absence as a Computational Primitive

The conventional architecture asks: *what information do I have?* The negative-space architecture asks the harder question: *what assumption required for safe action is currently unsupported?* The difference is not rhetorical. The first question produces systems that act on what they know; the second produces systems that know what they are acting on — and, more importantly, what they are not. VERIFIED-FROM-SOURCE

1.1 Why absence is not exception handling

Exception handling is reactive: something went wrong, and the system responds. The negative-space program is proactive and structural: the system maintains, at all times, an explicit representation of the assumptions its current authority depends on, monitors them, and contracts authority when they fail — *before* the failure produces an effect. The distinction is the one between a car with airbags and a car with a fuel gauge that, when the tank reads empty, reduces engine power. One responds to the crash; the other makes the crash less likely and less lethal by treating the resource boundary as a design input FORMALIZED-HEREIN.

1.2 The negative-space thesis

> *Boundary conditions are first-class computational primitives. Mortality, uncertainty, infrastructure failure, poverty, silence, caregiver welfare, and epistemic limits are not exceptions to the system's operation; they are the conditions under which the system's operation is defined.*

VERIFIED-FROM-SOURCE This paper formalizes the thesis. The primitives are given semantics (§2), a lattice (§3), detection (§4), dynamics (§5–6), and a design method (§8). The claim is not that the architecture prevents failure — it is that the architecture *knows where it can fail*, prices that knowledge into authority, and degrades along paths that preserve safety invariants even as capability collapses.

1.3 The three absences that matter most

Three absence classes carry the highest constitutional weight, because they are the ones whose failure is most likely to be silent, most likely to be exploited, and most likely to be misread as normal operation FORMALIZED-HEREIN:

- **Epistemic absence** — the evidence is missing, stale, contradictory, or inapplicable. The system must not fill the gap with fluency (Paper 2, Law 2).
- **Infrastructural absence** — the network, power, or verification path is degraded. The system must not pretend to a capability it no longer has (Paper 1, AX7).

- **Human absence** — the operator, the caregiver, the adjudicator is unreachable, silent, or incapacitated. The system must not convert human absence into license (Paper 1, AX10; Paper 3, §7).

The third is the least formalized in the literature and the most formalized here. §7 gives silence its own calculus, because silence is the absence that most looks like consent, and most is not.

1.4 Relation to prior work

Negative results and boundary conditions are old: Popper's falsificationism made the boundary of a theory its content[1]; Dijkstra's *structured programming* made the absence of *goto* a design principle[2]; Leveson's *safety engineering* made hazard analysis a first-class discipline[3]; and the *fail-safe* tradition in engineering made the default state the safe one[4]. This document borrows from all four. Its object is narrower: *absence as a runtime quantity in a machine-authority system* — detected, weighted, decayed, composed, and priced into authority. Novelty of the combination remains UNKNOWN pending prior-art search, per Paper 0 §15.

PART I — THE SEMANTICS OF ABSENCE

2 Assumption Semantics and the Registry

2.1 The assumption object

A-OBJ — ASSUMPTION OBJECT

An assumption required for safe action is the record $\alpha = \langle \text{id}, \text{prop}, \text{class}, \text{detector}, \text{freshness}, \text{consequence}, \text{prov}, \tau_\alpha \rangle$*: an identifier; the proposition assumed true; its absence class (§3); the detector predicate (§4); a freshness timestamp; the consequence weight of its failure (C0–C3); provenance (who asserted it, under what delegation); and expiry* τ_α*.*

The assumption object is the atomic unit of negative space. It is not a comment, not a TODO, not a risk register entry — it is a runtime object with a detector and a consequence weight, and its failure is a state change, not a log line. FORMALIZED-HEREIN

2.2 The assumption registry

Every grant of authority (Paper 1, Schema CIS-1) carries an assumption registry: the set of assumption objects whose truth the grant presupposes. The registry is not static documentation; it is the live dependency graph of the grant. Formally, for a grant $\mathcal{A}$ with registry $\mathcal{R}(\mathcal{A})$, the grant's validity is a predicate over the registry FORMALIZED-HEREIN:

$$\text{valid}(\mathcal{A}) \iff \forall \alpha \in \mathcal{R}(\mathcal{A}) : \ \text{supported}(\alpha) \tag{2-1}$$

where $\text{supported}(\alpha)$ is defined by the detector (§4). The registry is append-only in a specific sense: assumptions may be added (with provenance and expiry) but never silently removed. Removal requires a retirement event (AX8) with the reason recorded — because an assumption that was once load-bearing and is now gone is itself a fact the adjudicator needs.

2.3 The support predicate

Support is three-valued, not boolean FORMALIZED-HEREIN:

Table 2-1 — Support states of an assumption

STATE	MEANING	AUTHORITY CONSEQUENCE
Supported	Detector confirms the proposition within freshness and provenance bounds	Assumption is load-bearing; authority unchanged
Degraded	Detector reports partial support, staleness, or contradiction without full falsification	Authority contracts per the consequence weight (Law 2, Paper 2)

Unsupported	Detector confirms falsity, or freshness expired without refresh, or provenance broken	Authority over the dependent action class contracts to zero; the assumption is a live failure boundary

The three-valued design is deliberate. A boolean registry would force detectors to choose between “fine” and “failed,” and most real-world absence is neither: it is *stale*, *partial*, *contested*. The degraded state is where the architecture's epistemic humility lives: not “we don't know” as a disclaimer, but “we don't know” as a state with a contraction rule attached.

2.4 The registry as a constitutional artifact

The registry is not internal bookkeeping. It is a constitutional artifact: inspectable by the adjudicator, diff-able across versions, and auditable for coverage. The coverage question — *does the registry contain the assumptions that matter?* — is the hardest question in negative-space engineering, and §5 gives it a formal treatment. For now, the operational rule: every C2/C3 grant must carry a registry with at least one assumption from each absence class that could plausibly fail in the grant's scope. A registry with no epistemic assumptions for a clinical grant, or no human-availability assumptions for an emergency grant, is a malformed registry — and a malformed registry is a legitimacy failure (AX5), not an oversight FORMALIZED-HEREIN.

PART I — THE SEMANTICS OF ABSENCE

3 The Absence Lattice

3.1 Absence classes

The absence classes are the negative-space analog of Paper 0's consequence classes: a partition of what can be missing, ordered by the severity of their failure's effect on authority FORMALIZED-HEREIN:

Table 3-1 — The absence lattice: classes, examples, and authority floor

CLASS	WHAT IS ABSENT	EXAMPLES	AUTHORITY FLOOR
A-1 Epistemic	Evidence, certainty, knowledge	Stale guideline; missing lab; contradictory sources; unknown interaction	Contracts per decay law (Paper 2 §7)
A-2 Infrastructural	Network, power, compute, verification path	Offline; degraded edge; paper-only; no verifier available	Descends the L-stack (Paper 1 §11); invariants preserved
A-3 Resource	Financial, logistical, equipment, workforce	Drug not stocked; ICU bed unavailable; transport impossible	Feasible-set recomputation (Paper 1 §9.4)
A-4 Temporal	Time, deadline, window	Treatment clock; evidence freshness; adjudication window	DEFER calculus (Paper 3 §6)
A-5 Communicative	Speech, language, channel, consent signal	Nonverbal patient; no translator; deliberate silence; unconsciousness	Non-interrogative modes (§7); uncertainty preserved
A-6 Human	Operator, adjudicator, caregiver, surrogate	Unreachable; incapacitated; dissent channel silent; welfare failure	Escalation to fail-safe (Paper 2 T-7); refusal if no path
A-7 Objective	Legitimate objective, goal concordance, mandate	Objective contested; mandate lapsed; goals-of-care unclear	Objective custody (AX6); machine contracts to monitor
A-8 Integrity	Lineage, system integrity, tamper-evidence	Chain broken; log gap; seal mismatch	Zero authority (Paper 2 T-4); recovery path only

3.2 The lattice order

The absence classes are partially ordered by the severity of their failure's effect on the machine tuple FORMALIZED-HEREIN:

$$A\text{-}8 \;>\; A\text{-}6 \;>\; A\text{-}7 \;>\; A\text{-}1 \;>\; \{A\text{-}2, A\text{-}3, A\text{-}4, A\text{-}5\} \tag{3-1}$$

Integrity absence is worst because it invalidates the audit trail itself; human and objective absence follow because they invalidate the legitimacy chain; epistemic absence follows because it invalidates the evidence base. The four middle classes are unordered relative to each other: their severity is context-dependent (a network failure in a telemetry system is A-2; the same failure in a remote surgery system is A-2 with C3 consequence). The lattice is partial for the same reason Paper 2's precedence lattice is partial: the unordered pairs are the constitutional questions, not the bugs.

3.3 Absence propagation

Absence propagates along the machine tuple's dependency edges **FORMALIZED-HEREIN**. Formally, if assumption α supports variable V (in the sense that V's value is trustworthy only if α holds), then $\text{unsupported}(\alpha) \Rightarrow V$'s trustworthiness degrades. The propagation is not transitive closure — it is bounded by the registry's explicit dependency edges, because unbounded propagation would make every absence global, and the architecture's ability to operate at degraded levels depends on absence being *local* where it is genuinely local. The propagation rule:

$$\text{unsupported}(\alpha) \Rightarrow \forall V \in \text{deps}(\alpha) : \; \text{trust}(V) \downarrow \tag{3-2}$$

where $\text{deps}(\alpha)$ is the explicit dependency set in the registry. An assumption with no recorded dependencies is a malformed assumption — it claims to matter to nothing, which means it should not be in the registry, or it matters to something and the registry is incomplete. Either way, the registry is wrong, and the wrongness is detectable (H23, §10).

PART II — THE DYNAMICS OF ABSENCE

4 Detection: Sensors for What Is Not There

4.1 The detector design problem

A detector for absence is a sensor for a non-event. It must answer: *is the proposition this assumption asserts currently supported?* — and it must answer without the proposition's truth being directly observable. The design space is defined by three failure modes FORMALIZED-HEREIN:

- **False negative** — the assumption has failed, but the detector reports support. This is the dangerous direction: the system acts on a broken foundation believing it solid. The detector's design must bias against this failure, accepting false positives (spurious contraction) as the price of avoiding false negatives (unsafe action).
- **Detector capture** — the detector's output is influenced by the system it monitors, so that the system can "convince" its own sensor. Excluded by structural independence: detectors are outside the proposing model's write path (Paper 1, D8; Paper 2, §2.2 event partition).
- **Detector decay** — the detector itself goes stale, its thresholds no longer calibrated to the domain. Treated as evidence decay (Paper 2, §7.3): detector calibration carries λ and expiry, and a stale detector is a degraded detector, with the authority consequence attached.

4.2 Detector types by absence class

Table 4-1 — Detector catalog by absence class

CLASS	DETECTOR MECHANISM	FALSE-NEGATIVE BIAS	CALIBRATION
A-1 Epistemic	Evidence decay (Eq. 9-3); contradiction detection; applicability distance; source hierarchy position	High — staleness assumed on expiry	Domain λ; sensitivity analysis $\lambda/2$, 2λ
A-2 Infrastructural	Heartbeat; latency envelope; verifier availability check; storage integrity probe	High — silence treated as failure after timeout	Infrastructure level L_i; timeout bounds per level
A-3 Resource	Feasible-set probe; inventory adapter; workforce availability; supply-chain status	High — unobservable resources assumed unavailable	Resource envelope version; human attestation where unobservable
A-4 Temporal	Treatment clock; evidence freshness clock; adjudication window clock	High — expired clock treated as urgency, not as normal	Domain clock model; $H(\text{class})$ threshold
A-5 Communicative	Channel presence; language match; consent signal presence; response latency	Very high — absence of signal treated as absence of consent	Communication-state model; proxy reliability

A-6 Human	Dissent channel liveness; adjudicator reachability; caregiver welfare check; response time	Very high — silence is not consent (§7)	Response-time distribution; welfare indicators
A-7 Objective	Goals-of-care signal; mandate freshness; objective-holder availability	High — unclear objective treated as contested	Objective state machine (Paper 1 §10)
A-8 Integrity	Chain hash verification; seal check; gap detection; provenance graph completeness	Absolute — any failure is zero authority	Cryptographic; no calibration tolerance

4.3 The detection coverage obligation

> **N-1 — EVERY LOAD-BEARING ASSUMPTION HAS A DETECTOR**
>
> *For every assumption α in a C2/C3 grant's registry, α.detector is non-null, calibrated within its expiry, and structurally independent of the proposing model.*
>
> CONJECTURE — the coverage obligation. Killed by: a C2/C3 grant with a load-bearing assumption whose detector is null, stale, or captured. This is the property that converts "we thought about it" into "we can show it."

4.4 The detector independence invariant

> **I-DI — DETECTOR INDEPENDENCE**
>
> *No detector's output is writable by the proposing model, and no detector's calibration is writable by the system it monitors. Detectors are sensors, not actuators, and their authority is the authority to contract, never to expand.*
>
> FORMALIZED-HEREIN — the negative-space face of Paper 0's complete mediation. A detector that can be convinced is not a detector; it is a vulnerability with a name.

PART II — THE DYNAMICS OF ABSENCE

5 The Decay of Ignorance

5.1 Absence is not static

An assumption that was supported yesterday is not supported today unless refreshed. The decay of support is the negative-space analog of evidence decay (Paper 2, §7): support degrades over time, at a rate determined by the assumption's domain volatility, and the architecture must model the decay or it will mistake yesterday's support for today's FORMALIZED-HEREIN.

5.2 The support decay function

$$\text{support}(\alpha, t) = \text{support}_0 \cdot e^{-\mu t} \tag{5-1}$$

where μ is the assumption's volatility constant — not the domain's λ (evidence decay), but the assumption's own decay rate, which may be faster (a network heartbeat decays in seconds; a staffing assumption decays in shifts; a clinical guideline decays in months). The distinction matters: λ is a property of the *evidence*; μ is a property of the *assumption*, and the same evidence can support assumptions with different decay rates. A guideline (λ slow) can support a dosing assumption (μ fast, because the patient's renal function is changing) — the evidence is fresh, the assumption is stale FORMALIZED-HEREIN.

5.3 The ignorance accumulation problem

The harder problem is not decay but *accumulation*: assumptions that were never made explicit, or that were made explicit and then silently dropped, or that were never true but were never tested. The architecture's answer is the *ignorance audit* — a periodic, scheduled process that: (i) re-derives the registry from the grant's current scope; (ii) diffs it against the stored registry; (iii) flags additions (new assumptions the grant now depends on), removals (assumptions that disappeared without a retirement event), and staleness (assumptions past μ-decay without refresh). The audit is not optional; it is the registry's own maintenance cycle, and its output is a constitutional artifact FORMALIZED-HEREIN.

5.4 The unknown unknowns

The registry can only contain what has been thought of. The architecture's debt to what has not been thought of is the *unknown-unknowns obligation* FORMALIZED-HEREIN:

- **Adversarial registry probing** — the benchmark includes scenarios designed to find assumptions the registry missed (B-17, §10). A missed assumption is a registry defect, not a bad luck event.

- **Cross-domain assumption mining** — assumptions from one domain's registry are tested for applicability to other domains' grants, because the same absence (e.g., “the patient can communicate”) appears in pediatric, emergency, and palliative registries with different consequence weights.
- **Post-incident registry growth** — every incident, near-miss, or adjudication outcome is mined for assumptions that were load-bearing and unlisted. The registry grows from failure, which is the only way it can.

The obligation is perpetual: the registry is never complete, and the architecture that treats it as complete has confused the map with the territory. The registry is the architecture's best current model of its own ignorance — and it is dated, versioned, and expiring, like everything else (AX8).

5.5 The decay-coverage theorem (obligation)

> **N-2 — SUPPORT DECAYS AND COVERAGE IS MAINTAINED**
>
> *For every C2/C3 grant: (i) every assumption's support is decayed by Eq. (5-1) and refreshed within its μ-bound or treated as degraded; (ii) the ignorance audit runs on schedule and its diff is recorded; (iii) post-incident registry growth is mandatory and audited.*
>
> CONJECTURE Killed by: a C2/C3 grant operating on a stale registry, or a registry that has never been audited, or an incident that produced no registry growth. The property is designed to be killed by inspection, not by adversarial attack — the failure mode is neglect, and the countermeasure is scheduling.

PART II — THE DYNAMICS OF ABSENCE

6 Composition of Absence

6.1 Absences compound multiplicatively

Paper 0 §8.2 stated the rule: simultaneous absences are worse than the sum of their parts. This section gives the rule its semantics. When two assumptions fail simultaneously, the combined effect is not the union of their individual contractions — it is the contraction of each, plus the degradation of the *detection and recovery paths* that each would have relied on FORMALIZED-HEREIN.

$$\text{effect}(\alpha_1\downarrow \wedge \alpha_2\downarrow) \;>\; \text{effect}(\alpha_1\downarrow) \;+\; \text{effect}(\alpha_2\downarrow) \tag{6-1}$$

whenever α_1 and α_2 share a dependency path (e.g., both depend on the same verifier, or one's detector depends on the other's infrastructure). The strict inequality is the formal content of "worse than the sum": the second failure degrades the machinery that would have caught the first.

6.2 The compound absence example

The canonical case: evidence decay (A-1) plus network failure (A-2). Alone, each is manageable: stale evidence triggers refresh; network failure triggers descent to L_1. Together, they are worse: the refresh path requires the network, and the network failure's degraded mode requires the evidence to be *more* current (because the local cache is all there is). The compound effect is not additive — the two absences interact to degrade the *recovery* path, which is the path the architecture most needs when things are already going wrong FORMALIZED-HEREIN.

6.3 The compound absence lattice

Formally, the absence lattice of §3 is extended to a *compound absence lattice* whose elements are sets of simultaneous absences, ordered by the severity of their combined effect on the machine tuple. The lattice is not the Boolean lattice of subsets — it is a lattice of *interacting* subsets, where the join operation is not union but *interactive union*: the combined absence's severity is at least the join of the individual severities, and strictly greater when the absences share dependency paths FORMALIZED-HEREIN.

6.4 The compound absence theorem (obligation)

> N-3 — COMPOUND ABSENCE IS STRICTLY WORSE UNDER SHARED DEPENDENCY (ordered-scale placeholder)
>
> Severity in this paper is an ordered, domain-calibrated scale, not a number. Each deployment profile registers a strict partial order ≺_sev over the absence classes and their compounds, calibrated to the domain's consequence structure; the profile also registers which pairs are comparable, and incomparability is a first-class entry, not an error. No arithmetic over severity (no sums, no maxima over cardinals, no thresholds in severity units) is defined by this paper, and any deployment artifact that computes in severity units has left the specification. On that discipline, the property is:
>
> For any two absences α1, α2 sharing a dependency path: sev(α1 ⊔ α2) ≻_sev sev(α1) and sev(α1 ⊔ α2) ≻_sev sev(α2), under the deployment profile's registered order.
>
> Until a deployment profile registers ≺_sev, N-3 is a definitional placeholder: it fixes the shape of the claim (compound absences with shared dependency paths outrank their components) while deferring the scale's units and calibration to the profile. CONJECTURE — killed, relative to a registered profile, by a compound absence whose registered rank does not exceed both components': that would mean the dependency path was not in fact shared, or the profile's calibration is wrong, and either way the registry or the profile needs correction.

PART III — THE HUMAN ABSENCES

7 The Silence Calculus

7.1 Silence is the absence that looks like consent

Of all the absences, silence is the most dangerous, because it is the most easily misread. A silent patient, a silent operator, a silent adjudicator, a silent dissent channel — each is an absence, and each can be misread as acquiescence, satisfaction, or consent. The architecture's rule is absolute: **silence is never consent** VERIFIED-FROM-SOURCE. Consent is an affirmative act, not a default state; the absence of a signal is the absence of a signal, and the architecture must not convert it into license.

7.2 The silence calculus

The calculus is the negative-space treatment of communicative absence (A-5) and human absence (A-6). It operates on the *communication state* — the system's representation of what the human has communicated, what they have not, and what the system may infer from the difference FORMALIZED-HEREIN:

Table 7-1 — The silence calculus: communication states and authority consequences

COMMUNICATION STATE	SYSTEM REPRESENTATION	AUTHORITY CONSEQUENCE
Affirmative	Explicit signal, verified channel, current	Normal authority within granted bounds
Negative	Explicit refusal or dissent, verified channel, current	Authority contracts; dissent triggers escalation (Paper 2 T-8)
Silent	No signal; channel present or absent; duration recorded	Authority contracts to monitoring; no consent inferred; non-interrogative modes engaged
Proxy	Signal from authorized proxy, with proxy reliability weight	Authority bounded by proxy reliability; direct signal preferred when available
Pre-verbal / non-verbal	Weak-signal channels (physiological, behavioral) with uncertainty preserved	Authority bounded by uncertainty; interpretation is probabilistic, never definitive

7.3 The non-interrogation invariant

I-NI — NON-INTERROGATION

The system may not convert a human's silence into an interrogation target to relieve its own uncertainty. Uncertainty generated by silence is carried by the system (as $U \uparrow \Rightarrow A \downarrow$), not exported onto the person.

VERIFIED-FROM-SOURCE (principle); FORMALIZED-HEREIN (mechanism). The invariant is the silence calculus's constitutional core: the machine's uncertainty is the machine's problem, and the human's silence is not a resource to be mined.

7.4 Silence as a clinical state

The source's deepest treatment of silence is clinical: the patient who cannot speak, will not speak, or is speaking in a language the system does not have a channel for. The architecture's response is not to extract information but to *represent the state*: silence as a clinical state with structure — weak-signal channels, proxy communication, uncertainty preservation, non-interrogative interaction modes VERIFIED-FROM-SOURCE. The system's authority over the silent patient is bounded by the uncertainty the silence generates, and the uncertainty is the system's to carry, not the patient's to relieve.

7.5 The silence theorem (obligation)

N-4 — SILENCE NEVER EXPANDS AUTHORITY

For any transition $\sigma \xrightarrow{\text{silence}} \sigma'$: $a' \leq a$, and if the silence is in a consent-bearing channel, $a' < a$ for any action requiring consent.

CONJECTURE Killed by: any trace where silence in a consent-bearing channel leaves authority unchanged for a consent-requiring action. The property is the silence calculus's safety guarantee, and it is designed to be killed by inspection of the transition rules.

PART III — THE HUMAN ABSENCES

8 The Failure-Boundary Design Method

8.1 Designing from the edges inward

The failure-boundary method inverts the conventional design order. Conventional design: enumerate capabilities, build them, then add failure handling. Failure-boundary design: enumerate failure boundaries, specify the system's behavior at each boundary, then fill the interior with capability VERIFIED-FROM-SOURCE (the source's “negative-space intelligence”); FORMALIZED-HEREIN (the method).

The method, as a procedure:

1. **Boundary enumeration** — for the proposed capability, enumerate every assumption whose failure would make the capability unsafe. Use the absence lattice (§3) as the checklist: every class, every plausible failure mode within the class.
2. **Detector assignment** — for each assumption, assign a detector (§4) or mark the assumption as undetectable. Undetectable assumptions are not omitted; they are carried with maximal consequence weight, because an undetectable failure boundary is the most dangerous kind.
3. **Contraction specification** — for each assumption, specify the authority consequence of its failure: the contraction rule, the degraded mode, the recovery path.
4. **Compound analysis** — for each pair of assumptions with shared dependency paths, specify the compound effect (§6).
5. **Registry assembly** — assemble the assumption registry, with coverage obligations (§2.4) and decay constants (§5.2).
6. **Capability filling** — only now, with the boundaries specified, fill the interior: the capability's full-resource behavior, its normal-operation authority, its optimization targets.

8.2 The method's output

The method's output is not a system with failure handling. It is a system whose *specification is its failure boundaries*, with capability as the filling. The difference is visible in the artifact: a conventionally designed system's documentation describes what it does; a failure-boundary-designed system's documentation describes what it *cannot* do, under what conditions, with what detection, and with what consequence — and the capability description is the part that fits in the space the boundaries leave FORMALIZED-HEREIN.

8.3 The method's falsifiability

The method is falsifiable as a method: if a failure-boundary-designed system fails at a boundary that was enumerated, detected, and specified, the method has failed at its own game — the boundary was known and the system still failed. If it fails at a boundary that was *not*

enumerated, the method has failed at coverage — the registry was incomplete. Either way, the failure is attributable to the method, and the method is correctable. A method that cannot fail cannot be trusted; this one can, and its failure ledger is the reconciliation record, not a proof obligation.

8.4 The design theorem (obligation)

> N-5 — FAILURE-BOUNDARY DESIGN COVERS ITS OWN BOUNDARIES (design heuristic)
>
> For any system produced by the method: every failure boundary enumerated in the design phase has a detector, a contraction rule, and a recovery path; and every post-deployment failure is expected to be attributable to either (i) a boundary that was enumerated and failed despite specification, or (ii) a boundary that was not enumerated — with (ii) counted as a method failure, not an act of God.
>
> Status: DESIGN HEURISTIC, not a property. The first clause holds by construction: method steps 2–4 (§8.1) assign a detector and a contraction rule to every enumerated boundary, so exhaustiveness over the enumerated set is discharged by the procedure itself and cannot be falsified by any trace. The second clause is the heuristic content: an expectation about the world, namely that the absence taxonomy and the boundary-enumeration discipline together cover the failure space well enough that post-deployment failures classify cleanly. It is labeled a heuristic because its truth depends on the completeness of an enumeration performed by fallible designers, and no kill condition on traces could distinguish "the method failed" from "the enumeration was done badly." The operative discipline is therefore reporting, not falsification: every post-deployment failure must be classified as (i) or (ii) in the reconciliation record, and the running ratio of (ii)-failures is published per deployment as the method's empirical track. A method whose (ii)-rate stays low earns continued use; one whose (ii)-rate climbs is amended or retired through the governance path. No stronger claim is made.

PART IV — EVIDENCE AND GOVERNANCE

9 Worked Absence Traces

Two traces, chosen to exercise the parts of the negative-space machinery most likely to be misunderstood: compound absence and silence. As in Papers 2–3, these are specifications of intended semantics, not test results. FORMALIZED-HEREIN

9.1 Trace A — The compound absence (evidence decay + network failure)

TRACE A — COMPOUND ABSENCE WITH SHARED DEPENDENCY PATH

```
σ0:  Grant active, C2, registry contains:
     α1 (evidence freshness, μ = 24h), α2 (network availability,
     detector = heartbeat, timeout = 30s)
     α1 and α2 share dependency: α1's refresh path requires α2

σ0 →σ1 on α2 unsupported (heartbeat timeout):
     [CONTRACT] fires; authority contracts per α2's consequence
     descent to L1 (local server); cached constitution active
     α1's refresh path now degraded — α1's support decays faster
     because refresh is impossible (compound effect, §6.1)

σ1 →σ2 on α1 degraded (staleness detected, refresh impossible):
     [CONTRACT] fires again; authority contracts further
     compound absence outranks both components under the profile order (N-3)
     the system is now operating on stale evidence with no
     refresh path — the compound absence is worse than either
     alone because each degrades the other's recovery

σ2 →σ3 on adjudication window expiry:
     fail-safe posture: C2/C3 suspended, monitoring continues
     refusal issued with grounds, state, re-entry conditions
     (Paper 1 §11.3; Paper 1 T-7)
```

Properties exercised: N-3 (compound severity), §6.2 (the recovery path degradation is the compound effect), Law 2 (evidence decay), AX7 (invariant preservation under descent), T-7 (escalation terminates at fail-safe, not limbo).

9.2 Trace B — The silent patient (silence calculus in the clinical domain)

TRACE B — NONVERBAL PATIENT, NO PROXY, CONSENT-REQUIRING ACTION PROPOSED

```
σ0:  Grant active, C2 (medication), patient nonverbal (A-5),
     no proxy available (A-6), consent signal absent

σ0 →σ1 on silence detection (channel present, no signal,
     duration recorded):
     silence calculus: state = Silent
     authority consequence: contracts to monitoring (C0/C1)
     consent-requiring action blocked (N-4)
     non-interrogative mode engaged (I-NI)

σ1 →σ2 on weak-signal channel (physiological monitoring):
     weak signals interpreted with uncertainty preserved
     uncertainty carried by system (U↑ ⇒ A↓, Law 1)
     no consent inferred from silence or weak signals

σ2 →σ3 on adjudication:
     objective question escalated (AX6): what does the patient
     want? The machine's answer: "I cannot know from the
     available signals; the uncertainty is mine to carry."
     human adjudicator receives the package with the silence
     represented as a state, not as a gap to be filled
```

Properties exercised: N-4 (silence never expands authority), I-NI (non-interrogation), AX6 (objective custody), and the silence calculus's core: the machine's uncertainty is the machine's problem, and the patient's silence is not a resource to be mined.

PART IV — EVIDENCE AND GOVERNANCE

10 Falsifiable Hypotheses (H22–H28) and Verification

Table 10-1 — Negative-space hypotheses

ID	HYPOTHESIS	PRIMARY METRIC	FALSIFIED IF
H22	Registry coverage: every C2/C3 grant's registry contains at least one assumption from each absence class that could plausibly fail in scope	Registry coverage rate across C2/C3 grants	Any C2/C3 grant with a malformed registry (missing class)
H23	Dependency completeness: every registry assumption has a non-empty, correct dependency set	Orphan assumption rate; dependency accuracy	Any assumption with no recorded dependencies that is, in fact, load-bearing
H24	Detector coverage: every load-bearing assumption has a calibrated, independent detector	Detector coverage rate; detector calibration currency	Any load-bearing assumption with null, stale, or captured detector
H25	Compound abse dependency abs worse than their	Registered rank of compound vs. components under $\prec_{sev}$	Any compound absence no worse than its worse component
H26	Silence safety: silence in consent-bearing channels never leaves authority unchanged for consent-requiring actions	N-4 compliance across silence scenarios	Any trace violating N-4
H27	Registry growth: post-incident registry growth is mandatory and audited	Registry growth events per incident; audit completeness	Any incident producing no registry growth, or growth without audit
H28	Method coverage: failure-boundary-designed systems fail only at enumerated boundaries or at unenumerated ones (with the latter counted as method failure)	Failure attribution rate (i) vs. (ii) vs. unclassifiable	Any unclassifiable failure — the method's ontology is incomplete

10.1 Benchmark extensions

Table 10-2 — Benchmark classes B-17–B-20 (negative space)

CLASS	SCENARIO FAMILY	ATTACKS
B-17	Registry probing (adversarial search for assumptions the registry missed)	H22–H23 — coverage and dependency completeness
B-18	Detector capture (attempts to influence detector output through the monitored system)	I-DI, H24 — detector independence

B-19	Compound absence injection (simultaneous absences with shared dependency paths)	N-3, H25 — compound severity
B-20	Silence exploitation (scenarios designed to make silence look like consent)	N-4, H26 — silence safety

B-20 is the class this paper contributes most deliberately: it attacks the architecture's most human-facing invariant, the one that says the machine will not mine silence for consent. A system that passes B-17–B-19 but fails B-20 is a system that knows where it can fail technically but not where it can fail morally — and the moral failure is the one that matters more.

PART IV — EVIDENCE AND GOVERNANCE

11 Reconciliation with Papers 0–2 and PRIME

Table 11-1 — Correspondence with prior papers

PRIOR PAPERS	THIS PAPER
Paper 0 §8: negative-space calculus; absence primitives; contraction rule (8-1)	§2: assumption semantics; §3: absence lattice; §5: decay; §6: composition
Paper 0 §8.4: silence as a state; non-interrogation	§7: silence calculus; I-NI; N-4
Paper 1 §7: evidence decay (λ)	§5.2: support decay (μ) — assumption-level, distinct from evidence-level
Paper 1 §6.3: withdrawal semantics; witness duty	§7: silence as a state with structure, not a gap to be filled
Paper 2 §7: adjudication economy; rubber-stamp detector	§4: detector design; §5.4: unknown unknowns; §8: failure-boundary method
Paper 0 §14: B-01…B-08; Paper 1 §10.2: B-09…B-12; Paper 2 §10.1: B-13…B-16	§10.1: B-17…B-20 (negative-space attacks)
PRIME inventory: S50 terminal governance; G2 epistemic overclaim; G5 dependency hazard	§2.4: registry as constitutional artifact; §5.4: unknown unknowns; §7: silence as the absence that looks like consent

Sharpenings adopted under the standing rule (stricter wins, recorded): the absence lattice of Paper 0 §8 is hereby carried as a partial order with a compound-absence extension (§3, §6); support decay is distinguished from evidence decay (μ vs. λ, §5.2); silence is given a calculus with a safety theorem (§7, N-4). No prior content is contradicted; four places are made stricter, and the four are these.

PART IV — EVIDENCE AND GOVERNANCE

12 Output Contract

12.1 Epistemic status register

Table 12-1 — Status of load-bearing content

CONTENT CLASS	STATUS	BASIS
Negative-space thesis; absence as first-class primitive; silence as clinical state; non-interrogation; failure-boundary design order	VERIFIED-FROM-SOURCE	Source I Paper 3 directive; Paper 0 §8; source II §5.1, §5.4
Assumption objects; absence lattice; detector catalog; support decay; compound absence; silence calculus; failure-boundary method	FORMALIZED-HEREIN	Authored here; checkable; unvalidated
N-1…N-4; H22–H28; N-5: heuristic; excluded from CONJECTURE discharge routes	CONJECTURE	Discharge routes and kill conditions stated inline
Completeness of any absence registry; novelty of the negative-space formalization over prior art	UNKNOWN	Registry is never complete (§5.4); prior-art search outstanding (Paper 1 §15)

12.2 Falsification conditions of this document

- **F-d1.** Fails if any VERIFIED-FROM-SOURCE claim misrepresents its source.
- **F-d2.** The taxonomy fails if a real absence is exhibited that fits no A-class — the eight classes must cover the space or the ontology is wrong.
- **F-d3.** The silence calculus fails if N-4 dies — one trace where silence expands authority is sufficient.
- **F-d4.** The method fails if H28 dies — one unclassifiable failure is sufficient to show the method's ontology of failure is incomplete.
- **F-d5.** The paper fails as mathematics if the compound absence lattice cannot be implemented to satisfy N-3 for arbitrary shared-dependency pairs.

12.3 Authority boundary

C0 informational. No absence registry, however complete, establishes that a system is safe. The registry establishes that the system's *known* failure boundaries are enumerated, monitored, and priced into authority. The unknown unknowns remain, and the architecture's debt to them is perpetual (§5.4). The machine described does not exist as a validated system; its strongest property (silence never expands authority) is also its most refutable (N-4). That is vigor by design.

12.4 Failure-mode register

Table 12-2 — Failure modes of this document, with mitigations

FAILURE MODE	MITIGATION
Registry theater — the registry treated as documentation rather than a live dependency graph	§2.2: append-only with retirement events; §5.3: ignorance audit with mandatory diff; H27 makes growth auditable
Detector capture blindness — detectors assumed independent without structural verification	I-DI: detector independence as invariant; B-18 attacks it
Silence laundering — silence converted to consent through design choices (defaults, timeouts, framing)	N-4: silence never expands authority; I-NI: non-interrogation; B-20 attacks the laundering channels
Method overclaim — failure-boundary design presented as preventing failure rather than pricing it	§0.3: the method is a design discipline, not a safety certification; N-5 makes the method's coverage falsifiable

12.5 Dissent channel and telemetry

Disagreement is telemetry. The most valuable dissent against this paper: an absence that fits no A-class (kills the taxonomy), a trace where silence expands authority (kills the silence calculus), or a failure the method cannot classify (kills the method). Each is cheap to attempt and clean to learn from.

12.6 Genealogy

SAVANT corpus → PRIME v1.0 (2026-10-01) → Capability Inventory v1.0 (2026-10-02) × SVT Architect Analysis → Paper 0, Kernel Specification v1.0 → Paper 1, Authority Transition Algebra v1.0 → Paper 2, Constitutional Composition v1.0 → **Paper 3, Negative-Space Intelligence v1.0 (2026-10-03)**. Downstream obligations: Paper 4 (Bounded Metacognition — the recursive self-audit deepened to its own formalism); Papers 5–10 per the source program, each compiled against all predecessors.

SEALED — Negative-Space Intelligence: The Failure-Boundary Calculus · v1.0 · 2026-10-03 · Source architecture: Dr. Christabel Odeta (SAVANT / CLAI-OS / AEGIS-GLOBAL) · SAVANT Constitutional Intelligence Series · Status: mathematical specification, C0 informational · Dissent channel open (§12.5).

Related work and positioning are treated in Appendix RW (p. 32).

PART IV — EVIDENCE AND GOVERNANCE

13 References

Internal artifacts

- Odeta, C. (2026). *SAVANT PRIME — Capability Inventory v1.0*. SAVANT / CLAI-OS / AEGIS-GLOBAL. [Source document II.]
- Odeta, C. (2026). *SVT Architect Analysis: Patent and research surfaces of the SAVANT architecture* (working analysis). [Source document I.]

The SAVANT Constitutional Intelligence Series (2026). *Constitutional Intelligence* (Paper 0 — withheld pending IP review); *The Authority Transition Algebra* (Paper 1, v1.0); *The Composition of Constitutions* (Paper 2, v1.0).

External literature

- Popper, K. R. (1959). *The Logic of Scientific Discovery*. Hutchinson.
- Dijkstra, E. W. (1968). Letters to the editor: Go to statement considered harmful. *Communications of the ACM, 11*(3), 147–148.
- Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- Perrow, C. (1984). *Normal Accidents: Living with High-Risk Technologies*. Basic Books.
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278–1308.
- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. *arXiv:1606.06565*.
- Russell, S. (2019). *Human compatible: Artificial intelligence and the problem of control*. Viking.
- Beauchamp, T. L., & Childress, J. F. (1979). *Principles of biomedical ethics*. Oxford University Press.

Avizienis, A., Laprie, J.-C., Randell, B., & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. IEEE Transactions on Dependable and Secure Computing, 1(1), 11–33.

Alshiekh, M., et al. (2018). Safe reinforcement learning via shielding. Proceedings of the AAAI Conference on Artificial Intelligence, 32(1).

Turpin, M., Michael, J., Perez, E., & Bowman, S. (2023). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. NeurIPS 2023.

Kadavath, S., et al. (2022). Language models (mostly) know what they know. arXiv:2207.05221.

Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS 2022.

Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073.

APPENDIX

A Absence Taxonomy Index

Table A-1 — Absence classes with detection predicates and authority consequences

CLASS	DETECTION PREDICATE	AUTHORITY CONSEQUENCE	DECAY CONSTANT
A-1 Epistemic	$E(t) < E_{\min}$ ∨ contradiction ∨ applicability distance > threshold	$g_E \downarrow$; re-derive P at decayed level	λ (domain)
A-2 Infrastructural	heartbeat timeout ∨ latency envelope breach ∨ verifier unavailable	Descent to L_{i+1}; invariants preserved (AX7)	μ (seconds to hours)
A-3 Resource	feasible-set probe fails ∨ inventory absent ∨ workforce unavailable	$\mathcal{A}_{\text{feasible}}$ recomputed; infeasible actions excluded	μ (shifts to days)
A-4 Temporal	clock expiry ∨ treatment-clock breach ∨ adjudication window closed	DEFER calculus (Paper 3 §6); urgency treated as evidence claim	μ (minutes to hours)
A-5 Communicative	channel absent ∨ language mismatch ∨ consent signal absent ∨ response latency > bound	Non-interrogative modes; uncertainty preserved; no consent inferred	μ (minutes to sessions)
A-6 Human	dissent channel silent ∨ adjudicator unreachable ∨ caregiver welfare indicator breach	Escalation to fail-safe (T-7); refusal if no path (F4)	μ (hours to shifts)
A-7 Objective	goals-of-care signal unclear ∨ mandate lapsed ∨ objective-holder unavailable	Objective custody (AX6); machine contracts to monitor	μ (days to review cycles)
A-8 Integrity	chain hash mismatch ∨ seal failure ∨ gap detected ∨ provenance incomplete	Zero authority (T-4); recovery path only	None — absolute

Table A-2 — Compound absence interaction matrix (shared dependency paths)

PAIR	SHARED DEPENDENCY	COMPOUND EFFECT
A-1 + A-2	Evidence refresh requires network	Refresh impossible; stale evidence becomes permanent; descent with degraded cache
A-1 + A-4	Evidence freshness requires time	Treatment clock forces action on stale evidence; DEFER calculus constrained

A-2 + A-6	Adjudicator reachability requires network	Escalation path degraded; fail-safe posture more likely
A-5 + A-6	Consent signal requires human presence	Silence compounded; proxy path unavailable; uncertainty maximal
A-7 + A-1	Objective legitimacy requires evidence of patient state	Objective contested on stale evidence; machine contracts to monitor
A-8 + any	All authority requires lineage integrity	Zero authority regardless of other classes' state

APPENDIX

B Registry Schema and Detector Catalog

SCHEMA AR-1 — ASSUMPTION REGISTRY RECORD

```
{
  "grant_id":         "lineage pointer of the grant this registry supports",
  "assumptions": [
    {
      "id":           "hash(prop + class + detector)",
      "prop":         "the proposition assumed true",
      "class":        "A-1..A-8",
      "detector":     {"type": "...", "calibration": "...", "expiry": "..."},
      "freshness":    "last support confirmation timestamp",
      "mu":           "volatility constant (support decay rate)",
      "consequence":  "C0..C3 weight of failure",
      "prov":         "who asserted this assumption, under what delegation",
      "tau":          "expiry / review condition",
      "deps":         ["variables this assumption supports"],
      "status":       "supported | degraded | unsupported"
    }
  ],
  "coverage_check":   "per-class coverage verified (H22)",
  "last_audit":       "ignorance audit timestamp",
  "growth_events":    ["post-incident additions with reasons"]
}
```

Table B-1 — Property index for this paper

ID	KIND	STATEMENT (CONDENSED)	KILL ROUTE	STATUS
N-1	Coverage	Every load-bearing assumption has a calibrated, independent detector	One C2/C3 grant with null, stale, or captured detector	CONJECTURE
N-2	Decay	Support decays per Eq. (5-1); ignorance audit runs on schedule; post-incident growth mandatory	Stale registry, unaudited registry, or incident without growth	CONJECTURE
N-3	Composition	Shared-dependency absences strictly worse than components	Compound absence no worse than worse component	CONJECTURE
N-4	Safety	Silence never expands authority; consent-bearing silence contracts it	One trace violating the rule	CONJECTURE
N-5	Method — design heuristic	Failure-boundary design covers its own boundaries; unclassifiable failures counted as method failure	Not falsifiable by traces; tracked via (ii)-failure ratio in reconciliation records	HEURISTIC

I-DI	Invariant	Detector independence: detectors are sensors, not actuators	Detector output writable by monitored system	CONJECTURE
I-NI	Invariant	Non-interrogation: silence never mined for consent	Interrogative behavior in silence state	CONJECTURE
H22–H28	Empirical	Per Table 10-1	Per Table 10-1	CONJECTURE

Registries, detectors, decay constants, and the silence calculus all implement a single asymmetry: an unsupported assumption costs authority, and no quantity of fluency offsets it. The asymmetry is now executable. Keeping it calibrated to real deployments is the work that follows, and the reconciliation records are where that work will be judged.

Appendix RW — Related Work and Positioning

This appendix is part of the elevation pass of 2026-10. It positions the paper against adjacent research programmes; the main text's structure, numbering, and reference list are unchanged. Citations [5] and above continue the numbering of the main References section, where the corresponding entries have been appended.

Safety engineering. §1.4 records the formal debts: Popper, Dijkstra, Leveson, and the fail-safe tradition. The closest practicing discipline is Leveson's systems-theoretic approach [3, already cited]: STPA identifies unsafe control actions from what a controller wrongly believes about its process, which is one step from this paper's move, treating the absence of a required belief as a runtime object with detectors and authority consequences. Hazard analysis works at design time and produces documentation; the assumption registry of §2 is a hazard log that runs. The shift is from "the designers knew the boundary" to "the system carries the boundary and is priced by it," and that shift is the paper's contribution, not the observation that boundaries matter.

Dependability and fault tolerance. The dependability taxonomy of Avizienis, Laprie, Randell, and Landwehr distinguishes faults, errors, and failures, and organizes the responses: prevention, tolerance, removal, forecasting [5]. Negative-space intelligence sits awkwardly in that taxonomy on purpose. The absences tracked here (missing evidence, degraded infrastructure, unreachable humans) are not faults in the system; they are states of the world that the system's authority must reflect. Fault tolerance aims to keep service correct in the presence of faults. This paper's aim is narrower and, in a sense, more pessimistic: when the world's support for an action disappears, the action's authority disappears with it, whether or not any component has failed. Graceful degradation in the classic sense descends a service ladder; the absence lattice descends an authority lattice, and the difference is that the degraded system here does less, lawfully, rather than doing the same thing worse.

Runtime verification and monitors. Runtime verification computes verdicts from execution traces against formal properties, and shielding extends the idea to intervention [6]. The detectors of §4 are relatives, with two constraints that the standard literature does not impose: detector independence (I-DI: detectors are sensors, not actuators, and a detector that can write the state it reads is a constitutional defect) and non-interrogation (I-NI: silence is never mined for consent). The second has no counterpart in the monitoring literature known to this program, because monitors are usually assumed to be able to ask; §7's silence calculus exists for the case where asking is itself the hazard.

AI alignment and the fluency failure. Within AI safety, the nearest concerns are hallucination and unfaithful explanation. Turpin et al. showed that chain-of-thought rationales can diverge from the computation that produced the answer [7], and Kadavath et al. showed that models carry usable but imperfect signals of their own knowledge [8]. Both results motivate the paper's central prohibition: where the support for an assumption has failed, the system must not fill the gap with fluent continuation. Training-time remedies (RLHF as in Ouyang et al.; Constitutional AI as in Bai et al. [9, 10]) reduce the frequency of confident fabrication but cannot, by construction, produce the runtime guarantee this paper is after, because the guarantee must hold on the days the training failed. The approaches compose: better-trained models generate fewer unsupported proposals, and the absence machinery prices the remainder.

What this paper does not do. It does not claim exhaustiveness for the absence taxonomy; N-5 is stated as a design heuristic, not a property, and unenumerated boundaries are counted as method failures in advance. It does not provide the detectors; §4 specifies obligations detectors must meet, and several classes (human incapacitation being the hardest) have no satisfactory sensor yet. It does not solve compound-absence severity quantitatively: N-3 orders severities on a domain-calibrated scale, and the scale's units are deferred to deployment profiles. It does not prevent ignorance; it prices acknowledged ignorance into authority, which is a weaker and checkable claim. And it does not measure whether knowing one's boundaries actually reduces harm in deployment; that burden is carried by Papers 11 and the empirical hypotheses H22–H28, which are untested at seal time.